

Quelles sont les bonnes pratiques pour **une science reproductible** ?



RR
FR FR Reproducible
Research network





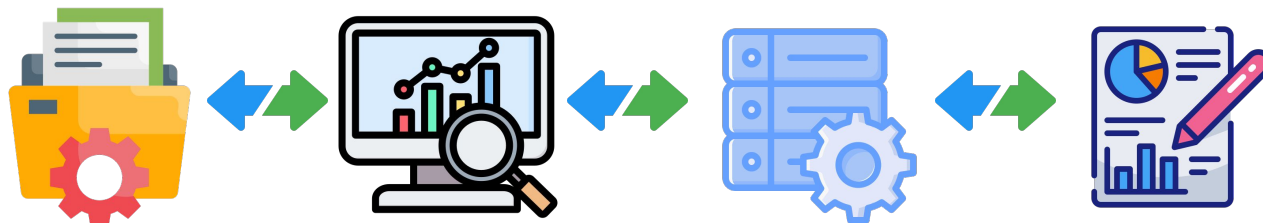
Maxime Dieudonné
PhD informatique

Ingénieur de recherche en
analyse de neuro-imagerie

maxime.dieudonne@univ-amu.fr

Mon travail consiste à

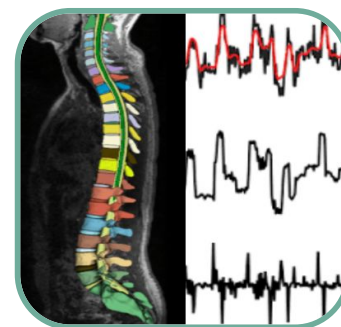
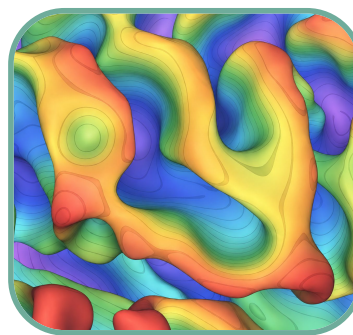
Gérer des données et les analyser à l'aide d'un ordinateur pour répondre à une question scientifique.



Sujets de recherche

- ▶ Comment voir si la maladie de parkinson affecte mon cervelet ?
- ▶ Comment le cerveau se plisse avec l'âge ?
- ▶ Comment les ordres du cerveau arrivent en bas du dos ?

Type de données



Photos et des vidéos du cerveau, et de la moelle épinière (Images IRM anatomiques et fonctionnelles)



Un Changement de Paradigme Scientifique

La science est entrée dans l'ère du **data-driven** : explosion des données, pipelines complexes, machine learning. *The Fourth Paradigm* (Hey et al., 2009) documente ce basculement. Le code est devenu un **instrument scientifique** au même titre que le microscope. Il doit être calibré, documenté et partageable.

EEG

20 à 50 étapes de traitement,
multiples bibliothèques Python

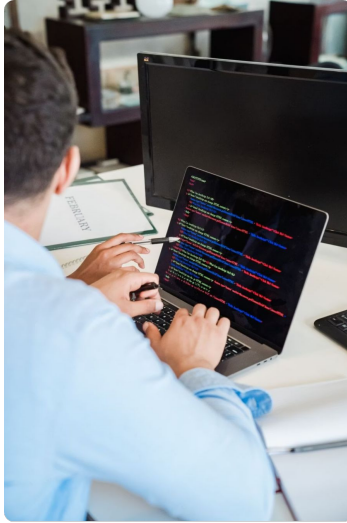
IRMf

Téraoctets de données, FSL,
FreeSurfer, fMRIPrep

Modélisation

DDM, bayésien: milliers de lignes de code

Des Chercheur·es devenu·es Développeur·es... Sans le Savoir



Le constat

La majorité des chercheurs écrivent leur propre code **sans formation dédiée** souvent seuls, sans tests ni documentation systématique.

Hannay et al. (2009)

Les scientifiques sont de facto des développeurs non formés.

Wilson et al. (2014)

Le code scientifique est rarement testé ; les erreurs logicielles sont une source majeure d'erreurs scientifiques.

Quelles sont les bonnes pratiques pour **une science reproductibles** ?

► Des résultats chiffrés reproductibles



ex : figures de l'article, métriques

Ne la joue pas solo !

- Science Ouverte
- Principe FAIR : Findable Accessible Inter-operable Reproducible

► Des conclusions reproductibles



ex : la difference de mes facteurs expérimentaux explique la difference de mes résultats.

Justifie que ce n'est pas du Pif !

- méthode (Principe de réfutabilité..)
- statistique (Model Nul, tests positif/négatif ..)

► Un travail reproductible



ex : Je peux améliorer mon code 6 mois plus tard

Organise toi comme un dev !

- méthode (Principe de réfutabilité..)
- statistique (Model Nul, tests positif/négatif ..)

Quelles sont les bonnes pratiques pour **une science reproductibles** ?

Esprit communautaire

- Des résultats chiffrés reproductibles



ex : figures de l'article, métriques

Ne la joue pas solo !

- Science Ouverte
- Principe FAIR : Findable Accessible Inter-operable Reproducible

Rigueur scientifique

- Des conclusions reproductibles



ex : la difference de mes facteurs expérimentaux explique la difference de mes résultats.

Justifie que ce n'est pas du Pif !

- méthode (Principe de réfutabilité..)
- statistique (Model Nul, tests positif/négatif ..)

Gestion de projet

- Un travail reproductible



ex : Je peux améliorer mon code 6 mois plus tard

Organise toi comme un dev !

- méthode (Principe de réfutabilité..)
- statistique (Nul Model, tests positif/négatif ..)



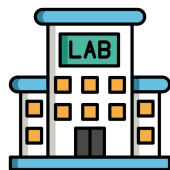
Discipline

Esprit communautaire - Rigueur scientifique - Gestion de projet



Acteur

Logiciels - Connaissances - Bonnes pratiques

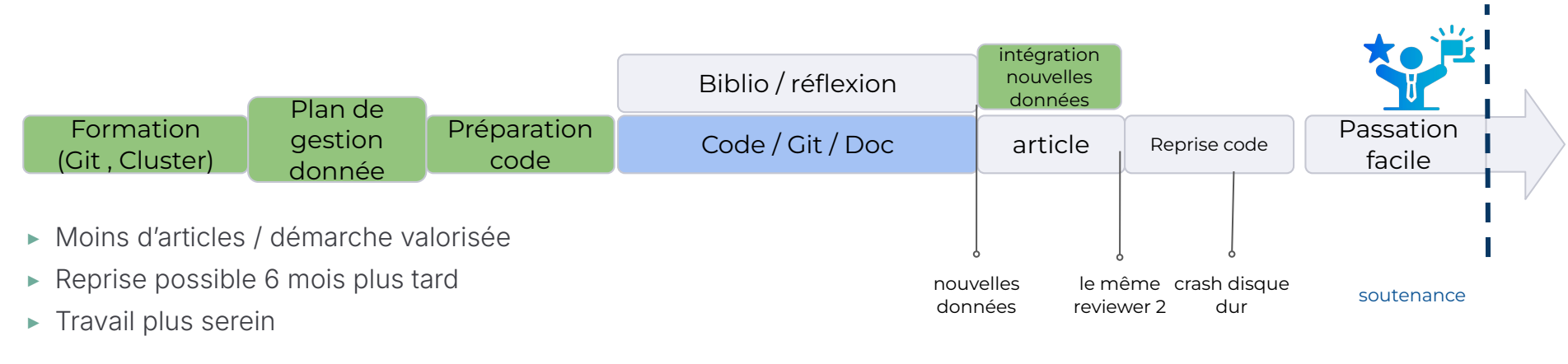
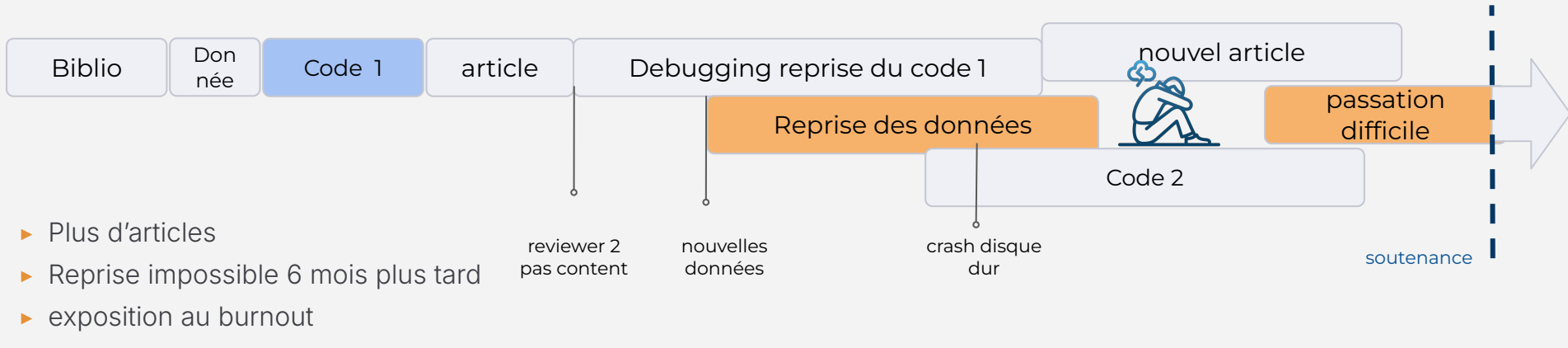


Centre

Vision - Catalogue de Formations - Cadre - Valorisation

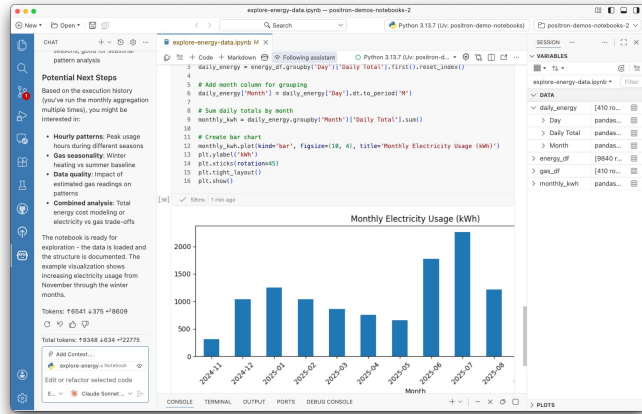
Il n'y a pas de reproductibilité possible sans un environnement de travail qui ne vous permet pas de vous organiser et de planifier la gestion du code et des données.

Gestion collective du travail



Jupyter Notebook

- ▶ Doc, code et figures centralisés
- ▶ Lecture séquentielle
- ▶ Joli / interactif



IDE (codium)

- ▶ Doc, code et figures modulaires
- ▶ Lecture non séquentielle
- ▶ non attractif pour non codeur

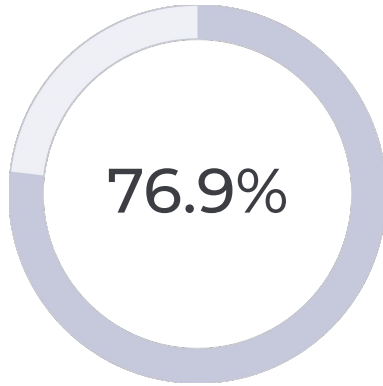


The screenshot shows the Codium IDE environment. The main area displays a code editor with the following Python code:

```
1 """
2 Crop anatomique T2w - Section 1.c du pipeline SpinalFlow.
3
4 crop_anat() - masque cylindrique 80mm depuis SegT2 + crop XY/Z
5
6 data_inspection
7
8 Prérequis : SegT2 produit par deepseg-t2 (resample + set deepseg_sc_lumbar_t2).
9 """
10
11 from typing import Dict
12
13 from function.utils.paths import ProjectPaths
14 from function.utils.run import get_cmd, check_output
15 from function.preprocessing.segmentation.cropCF import cropCF
16
17 def crop_anat():
18     """
19     Crop anatomique T2w en deux étapes (SegT2 produit par deepseg-t2) :
20     1. Masque cylindrique 80mm centré sur le C50 de SegT2
21     2. Crop XY puis Z (craffé @ sous le masque modulaire)
22     """
23     cropCF = section(crop) de fichier de config (PipelineConfig).cropCF.
24
25     cfg = cropCF or {}
26     mask_fm = cropCF.get('anat_mask_fm', 80)
27     offset_x = cropCF.get('anat_offset_x', 50)
28
29     anat = Files["T2"][[sub] for sub in sub]
30     seg_img = Files["DeepSegT2"][[sub] for sub in sub]
31     crop_img = Files["CropT2"][[sub] for sub in sub]
32
33     output_dir = os.path.join(OUTPUT_PATH, "anat_flow_analysis")
34     os.makedirs(output_dir, exist_ok=True)
35
36     # Run the pipeline
37     cmd = "python main.py --anat_mask_fm {mask_fm} --anat_offset_x {offset_x} --output_dir {output_dir}"
38     check_output(cmd)
```

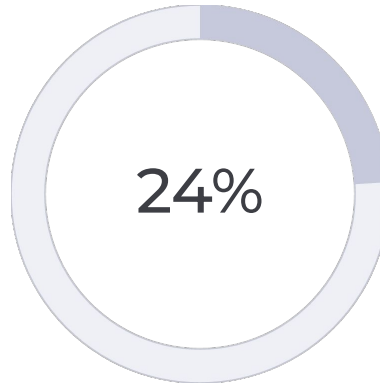
Jupyter : oui mais ...

Les Jupyter notebooks, outil de prototypage, sont souvent utilisés comme outil de publication, sans les pratiques garantissant la reproductibilité.



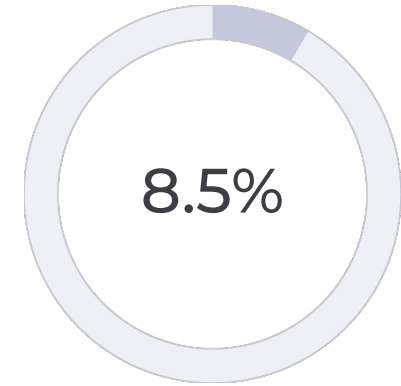
Sauts d'exécution

Notebooks avec cellules exécutées hors ordre
(Pimentel et al., 2019)



S'exécutent sans erreur

Sur 800 000 notebooks GitHub analysés



Reproduisent les résultats

Notebooks biomédicaux associés à des publications réelles (GigaScience, 2024)

⚠ Causes identifiées : dépendances obsolètes, versions différentes, environnement non documenté, ordre d'exécution non reproductible.

- ▶ Collaborer et versionner
- ▶ Organiser son code
- ▶ Gérer l'environnement
- ▶ Bien rédiger son code
- ▶ Assurer la robustesse
- ▶ Documenter



Git : un journal de bord scientifique



Un historique Git bien tenu permet de retrouver à quelle version du code correspond chaque figure d'un article.

Créer branche

Développer et tester

Commit sémantique

Merge après validation

Commits sémantiques (Conventional Commits)

feat

Nouvelle fonctionnalité

fix

Correction de bug

refactor

Amélioration du nommage

doc / test

Les idées à garder en tête pour aider à l'organisation de son code :

Je sais pourquoi j'organise mon code :

- Je souhaite retrouver facilement un fichier, un paramètre, 6 mois plus tard
- Je souhaite faciliter le travail de lecture d'un·e collaborateur·rice

Organiser son code se planifie :

- Je pense à l'organisation de mon code avant de coder !
- Je fais des pauses régulières pour réorganiser si besoin en fonction de l'avancé du projet
- Je ne cherche pas l'organisation parfaite car le projet évolue

2. Organiser son code

```
mon_projet/
├── data/                                # Données brutes (jamais modifiées)
│   ├── raw/
│   └── derivatives/
├── preprocessing
│   ├── functions/                       # Code source principal (fonctions, classes)
│   │   ├── __init__.py
│   │   ├── function_01.py
│   │   └── functions_02.py
│   ├── scripts/                         # Scripts d'orchestration (appelle fonctions/)
│   │   ├── run_function_01.py
│   │   └── run_function_02.py
│   └── utils/                           # fonctions utilitaires
│       ├── paths.py
│       ├── files.py
│       ├── config.py
│       ├── cli_arg.py
│       ├── run.py
│       └── subjects.py
├── exploration/                         # dossier dédié à l'exploration
│   ├── data_exploration/ ...
│   └── step_pipeline_exploration/ ...
├── figures/                             # Figures générées automatiquement
├── tests/                               # Tests unitaires
├── docs/                                # Documentation
├── config.toml                          # Paramètres du projet
├── environment.yml                      # Environnement conda reproductible
├── README.md                            # Point d'entrée du projet
└── .gitignore
```

Séparer logique, orchestration, figures et doc

- **Logique** : les calculs, traitements indépendants des données
- **Orchestration** : application des calculs sur nos données
- **Figures** : génération et sauvegarde automatique
- **Doc** : fichiers markdown.

Créer un dossier d'utilitaires

- **paths.py** : Gestion des chemins du projet.
- **files.py** : Construction d'un dictionnaire Nom ↔ Path.
- **config.py** : Chargement des paramètres de config.
- **cli_arg.py** : Argument CLI partagés par toutes les fonctions
- **run.py** : Utilitaire d'exécution de commandes externes
- **subjects.py** : Registre des sujets (lu avant le run)

Créer des dossiers spécifiques à l'exploration

Rester agile, ne pas mélanger l'exploration à la partie du pipeline validée.

UV combine 2 concepts :

- La gestion automatisée et intelligente des dépendances et sous-dépendances.
- La création d'un environnement virtuel isolé pour votre projet.

<https://openclassrooms.com/fr/courses/8445356-gerez-votre-environnement-python-avec-uv/8465886-installez-uv-et-lancez-votre-premier-environnement-virtuel>

Table des matières

PARTIE 1

Utilisez Poetry pour piloter votre environnement Python

1

Tirez un maximum de ce cours

2

Installez UV et lancez votre premier environnement virtuel

3

Prenez en main les fonctionnalités supplémentaires d'UV

Quiz : Gérer son environnement Python avec UV

Déclarez des packages

Lancer dans votre terminal la commande :

```
uv init nom_dossier_de_votre_choix
```

Cela va créer dans votre répertoire actuel, un nouveau dossier avec le nom que vous avez spécifié. Au sein de ce dossier, vous allez trouver 3 fichiers :

- Un [README.md](#), exactement le même que dans un répo Git. Il a le même objectif et ne vous fait que gagner un peu de temps au lieu de le créer par vous-même.
- Un script python `main.py`, qui n'est en réalité qu'un script générique, ne contenant pas grand-chose.
- Un fichier `pyproject.toml`. C'est le fichier le plus important parmi les 3 et le plus central dans votre usage d'UV.

Si vous ouvrez ce fichier `pyproject.toml` (avec un VSCode par exemple), vous verrez une structure comme la suivante :

```
[project]
name = "nom-dossier-de-votre-choix"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.12"
dependencies = []
```

Un code se lit plus souvent qu'il ne s'écrit

Deux outils de référence en Python

Flake8 : Linter

Détecte erreurs de style, variables inutilisées, imports manquants. Signale sans corriger.

Black : Formatter

Reformate automatiquement selon des conventions strictes. On ne discute plus du style.

Pre-commit hook : Black s'applique automatiquement à chaque commit Git sans y penser.

3. Gérer l'environnement

Problème d'espacement E231

Code

```
def bpm(beats,seconds):  
    return beats/seconds*60
```

Flake8

```
avant.py:1:11: E231 missing whitespace after ','
```



Correction

```
def bpm(beats, seconds):  
    return beats / seconds * 60
```

Import inutilisé F401

```
import numpy as np  
amplitudes=[12.3,15.7,9.8,20.1]  
moyenne = sum(amplitudes)/len(amplitudes)  
print( "Moyenne EEG:",moyenne )
```

```
avant.py:1:1: F401 'numpy as np' imported but unused  
avant.py:2:11: E231 missing whitespace after ','  
avant.py:2:16: E231 missing whitespace after ','  
avant.py:2:21: E231 missing whitespace after ','  
avant.py:4:7: E201 whitespace after '('  
avant.py:4:22: E231 missing whitespace after ','  
avant.py:4:33: E202 whitespace before ')'
```



```
amplitudes = [12.3, 15.7, 9.8, 20.1]  
moyenne = sum(amplitudes) / len(amplitudes)  
print("Moyenne EEG:", moyenne)
```

Problème d'espacement E225

```
def alerte(spo2):  
    if spo2<92:  
        return True  
    else:  
        return False
```

```
avant.py:2:13: E225 missing whitespace around operator
```



```
def alerte(spo2):  
    if spo2 < 92:  
        return True  
    else:  
        return False
```

Installation

```
pip install flake8 black
```

Commandes typiques flake8

```
# Vérifier un seul fichier
flake8 mon_fichier.py

# Vérifier tout un dossier / projet
flake8 .

# Vérifier plusieurs fichiers précis
flake8 fichier1.py fichier2.py

# Avec un fichier de config (ligne max, exclusions, etc.)
flake8 --max-line-length=100 .

# Ignorer certaines erreurs ponctuellement
flake8 --extend-ignore=E203,W503 .
```

Commandes typiques Black

```
# Reformater un seul fichier (modifie le fichier directement)
black mon_fichier.py

# Reformater tout un dossier / projet
black .

# Vérifier sans modifier (utile en CI/CD)
black --check .

# Voir les changements proposés sans les appliquer
black --diff mon_fichier.py

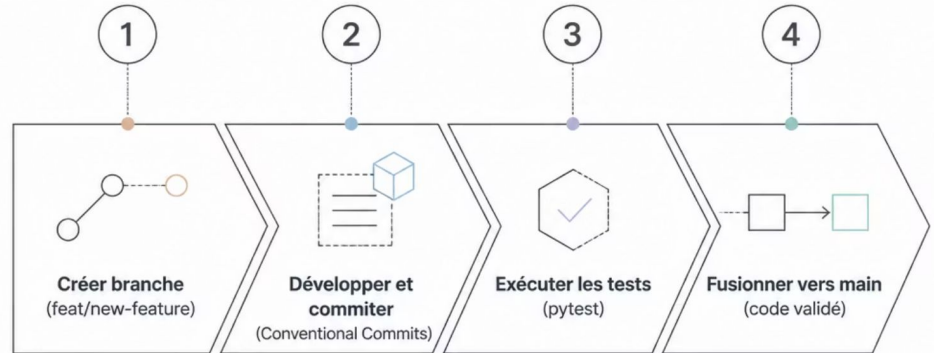
# Imposer une longueur de ligne personnalisée
black --line-length 100 .
```

Pourquoi tester ?

Qui valide le code qui fait tourner les statistiques ? Les **tests unitaires** (pytest) vérifient chaque fonction sur des exemples contrôlés et détectent les régressions immédiatement.

Git : non optionnel

Git trace chaque modification, relie un résultat publié à la version exacte du code. Les **branches** isolent le développement ; les **commits sémantiques** (feat:, fix:, doc:) rendent l'historique lisible.



Trois niveaux :

- README.md (porte d'entrée),
- commentaires (expliquent le pourquoi),
- docstrings (génèrent la doc HTML via Sphinx, Mkdocs).



GitLab institutionnel

Hébergé sur les serveurs du labo
confidentialité des données de santé.
CI/CD : tests automatiques à chaque
push.



Docker & Singularity

Encapsule l'environnement complet
dans une image portable. Singularity
pour HPC sans droits root.



SemVer + Zenodo

MAJEUR.MINEUR.CORRECTIF (1.0.0 →
1.1.0 → 1.1.1). Chaque version reçoit un
DOI citable.

Le code constitue votre preuve scientifique

Le code est désormais demandé par des journaux comme Nature Com.

La planification, une équipe impliquée, et la valorisation de la méthode sont essentielles à la reproductibilité.

un skill à retenir : séparer la logique de l'orchestration.

Contact : maxime.dieudonne@univ-amu.fr

Ressources

[The Turing Way](#) : Science reproductible

[Software Carpentry](#) : Formations pour chercheurs

[MNE-Python](#) : Projet scientifique bien structuré

[Cookiecutter Data Science](#) : Template de projet